



UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA

Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

Laboratorio No. 10 · Autorización segura: RBAC e IDOR/BOLA · API_TEATRO

ESPACIO ACADÉMICO:

Seguridad en el Desarrollo de Software

PRESENTADO A:

Sergio Arley Puerto Moreno

AUTORES:

Juan Sebastián Bonilla León

Juan Camilo Calderón Delgado

Silvana Sofía Siza Soriano

Cristian Emanuel Hernández Araque

Cristian Rodrigo Amaya Torres

2026

1. El problema: saber quién eres no es lo mismo que tener permiso

Hasta el Laboratorio 9 la API ya sabía dos cosas: qué aplicación la usaba (la API Key) y qué persona había iniciado sesión (el JWT). Pero el rol que viajaba en el token no servía para nada. Un asistente con sesión podía llamar a los mismos endpoints que un administrador, y cualquiera podía pedir las boletas de otra persona con solo cambiar el número en la URL.

En este laboratorio respondimos la pregunta que faltaba: **¿tienes permiso para hacer esto, y sobre este recurso en particular?** Lo trabajamos en tres partes:

- **6A · RBAC:** que el rol decida qué operaciones puede hacer cada usuario.
- **6B · Gestión de usuarios:** una forma controlada de crear cuentas de taquilla y de administración.
- **6C · IDOR/BOLA:** que cada asistente solo pueda ver y tocar sus propias boletas, aunque cambie el id.

La diferencia entre los dos códigos de error resume todo el laboratorio:

Código	Significa
401 Unauthorized	No sé quién eres: no hay sesión válida
403 Forbidden	Sé quién eres, pero no tienes permiso para esto

Cómo hicimos las pruebas

Arrancamos el servidor desde cero y preparamos los datos con un script: dos asistentes, A (`asistenteId 6`, boleta 8) y B (`asistenteId 7`, boleta 9), y una cuenta de taquilla. Luego lanzamos cada petición contra la API desde el navegador. Cada figura muestra el método, la ruta, quién hace la petición, el cuerpo enviado y la respuesta real del servidor con su código.

En ninguna figura aparecen la API Key, el `JWT_SECRET` ni la contraseña del administrador: la contraseña se ve como `.....` y los tokens recortados como `eyJhbGci....`. En las respuestas con listas largas, la captura va recortada por abajo.

2. Bloque 6A: el rol empieza a decidir

El middleware autorizarRoles

Creamos `src/middlewares/roles.middleware.js`. Recibe la lista de roles permitidos y hace dos preguntas en orden: si no hay usuario autenticado responde 401, y si el rol no está en la lista responde 403.

```
03 src/middlewares/roles.middleware.js archivo completo

QUÉ MUESTRA El middleware de RBAC: 401 si no hay usuario, 403 si el rol no corresponde

1 // =====
2 // Autorizar roles
3 // Responde a "¿tu rol puede hacer esta operación?"
4 // =====
5 // Va siempre después de autenticarJWT, que es quien deja req.usuario. La
6 // distinción entre los dos códigos es la del bloque: 401 es "no sé quién eres",
7 // 403 es "sé quién eres y no te corresponde".
8 const autorizarRoles = (...rolesPermitidos) => {
9   return (req, res, next) => {
10     // =====
11     // Debe existir usuario autenticado
12     // =====
13     if (!req.usuario) {
14       return res.status(401).json({ mensaje: "Usuario no autenticado" });
15     }
16
17     // =====
18     // Verificar rol
19     // =====
20     if (!rolesPermitidos.includes(req.usuario.rol)) {
21       return res.status(403).json({
22         mensaje: "No tiene permisos para realizar esta operación"
23       });
24     }
25
26     next();
27   };
28 };
29
30 // =====
31 // Exportaciones
32 // =====
33 module.exports = autorizarRoles;
34
```

401 y 403 responden preguntas distintas: «no sé quién eres» frente a «sé quién eres y esto no te corresponde».

Figura 1. `src/middlewares/roles.middleware.js`: 401 si no hay usuario y 403 si el rol no está permitido.

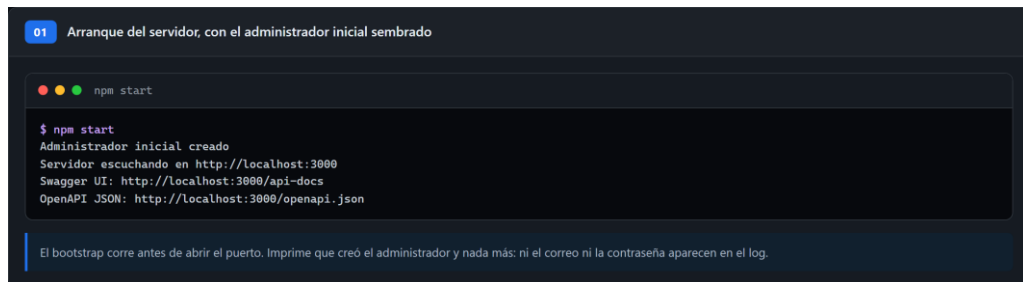
En cada ruta protegida la cadena quedó en este orden: primero `autenticarJWT` (¿quién eres?), luego `autorizarRoles` (¿puedes hacer esto?), después los validadores y por último el controlador. Así, una petición sin permiso ni siquiera llega a validar los datos.

autenticarJWT → autorizarRoles → validador → validar → controlador

¿Quién crea al primer administrador?

Aquí apareció un problema: solo un administrador puede crear otros administradores, pero al principio no existe ninguno. No quisimos abrir el registro público para eso, porque sería regalarle el sistema a quien llegue primero. La solución fue un **bootstrap**: al arrancar, el servidor crea el administrador inicial con los datos del `.env`, antes de abrir el puerto.

En `.env.example` solo van marcadores (`ADMIN_NOMBRE`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`). Los valores reales viven en el `.env`, que no se sube a GitHub, y al arrancar el servidor solo avisa que creó el administrador, sin mostrar el correo ni la contraseña:



```
01 Arranque del servidor, con el administrador inicial sembrado

$ npm start
Administrador inicial creado
Servidor escuchando en http://localhost:3000
Swagger UI: http://localhost:3000/api-docs
OpenAPI JSON: http://localhost:3000/openapi.json

El bootstrap corre antes de abrir el puerto. Imprime que creó el administrador y nada más: ni el correo ni la contraseña aparecen en el log.
```

Figura 2. Arranque del servidor: el administrador inicial se crea antes de abrir el puerto, sin imprimir credenciales.

Quién puede hacer qué

Aplicamos el RBAC a los cinco recursos de la API. La política quedó así:

- **Cartelera (eventos, localidades y funciones):** consultarla es público; crear, modificar, borrar o cambiar el estado de una función es solo del administrador.
- **Asistentes:** el administrador y la taquilla los consultan y los crean; modificarlos o borrarlos es solo del administrador. Cada asistente ve su propio perfil en `/api/asistentes/mio`.
- **Boletas:** el administrador lista todas; la taquilla atiende a cualquier comprador; el asistente solo ve las suyas, por `/api/boletas/mias` o por id si le pertenecen. Borrar boletas es solo del administrador.
- **Usuarios de taquilla y administración:** solo los crea el administrador.

La cartelera sigue siendo pública

Esta fue una decisión a propósito. Un teatro existe para publicar su cartelera, y nuestra página web (la Sala) la muestra sin pedir cuenta. Por eso los GET de eventos, localidades y funciones no piden JWT. Siguen protegidos por la API Key: quitamos la segunda barrera, no la primera.

Probando el RBAC

Los datos personales sí exigen sesión. Sin token la respuesta es 401:

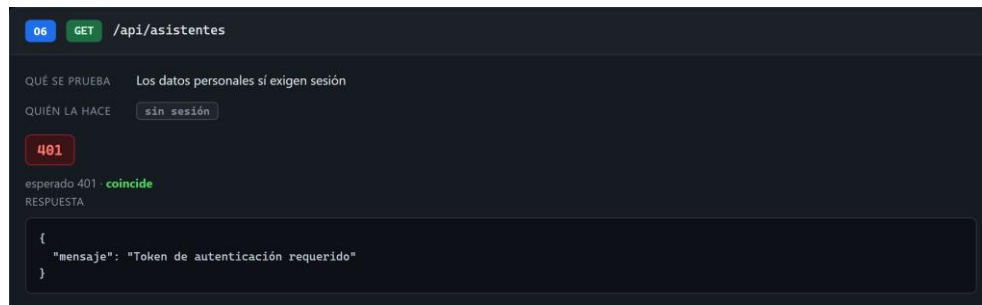


Figura 3. GET /api/asistentes sin token: 401 "Token de autenticación requerido".

Y si alguien modifica el token, la firma deja de coincidir y también recibe 401:



Figura 4. GET /api/asistentes con un JWT con la firma alterada: 401 "Token inválido".

Con un token válido, el rol decide. Un asistente recibe 403 porque su rol no tiene permiso para listar asistentes; el administrador, con la misma petición, recibe 200 con la lista. Lo mismo pasa si un asistente intenta crear un evento: 403, aunque el cuerpo sea válido, porque la petición se rechaza antes de llegar al validador.



Figura 5. GET /api/asistentes como asistente: 403. El token es válido, pero el rol no tiene permiso.

El registro público sigue sin dejar escalar

Volvimos a atacar el registro público como en el Laboratorio 8, ahora con cuatro campos a la vez: `rol: administrador`, `activo: false`, `passwordHash` y `esSuperAdmin`. Respondió 201, pero la cuenta quedó como asistente y activa. Ninguno de los campos colados tuvo efecto, porque la entrada se filtra por lista blanca.

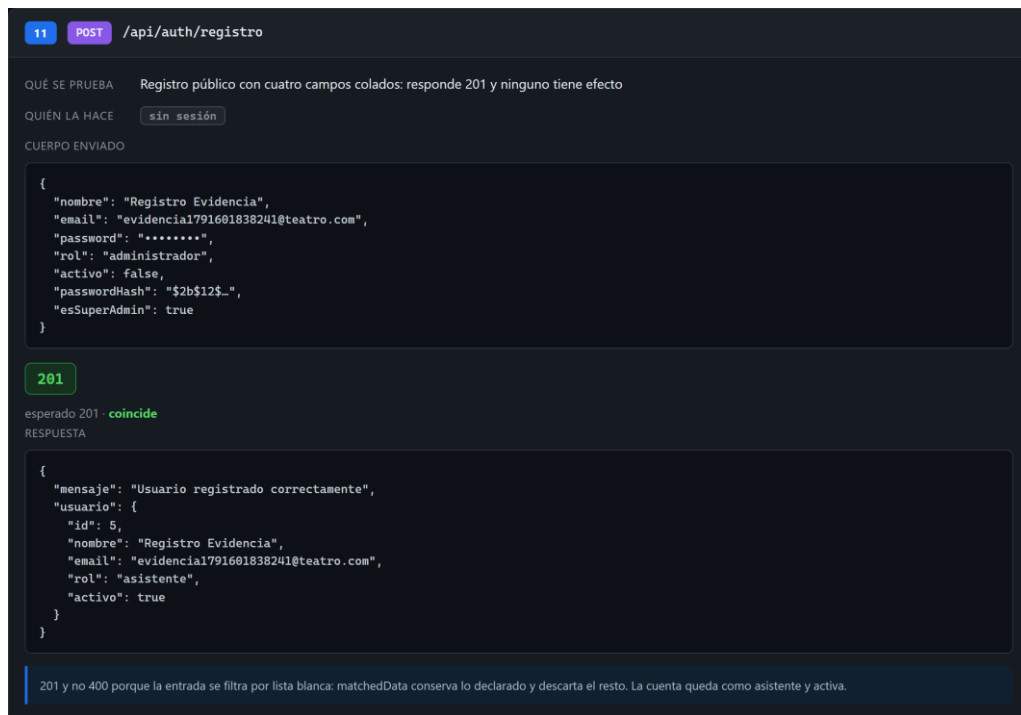


Figura 6. Registro con rol, activo, passwordHash y esSuperAdmin colados: 201, la cuenta queda como asistente y activa.

3. Bloque 6B: crear cuentas de taquilla y de administración

Ahora hay dos caminos para crear usuarios. El registro público, `POST /api/auth/registro`, lo puede usar cualquiera y siempre crea asistentes: el rol lo decide el servidor. El nuevo `POST /api/usuarios` solo lo puede usar el administrador, y ahí sí se acepta el rol desde el cuerpo, porque la operación ya está protegida. Un campo no es inseguro por existir en la petición; el problema es quién está autorizado a controlarlo.

Para no debilitar el registro público no tocamos `crearUsuario()`, que sigue forzando el rol asistente. Creamos una función aparte, `crearUsuarioAdministrativo()`. Si hubiéramos hecho una sola función con el rol opcional, un descuido en una ruta futura se convertiría en escalada de privilegios.

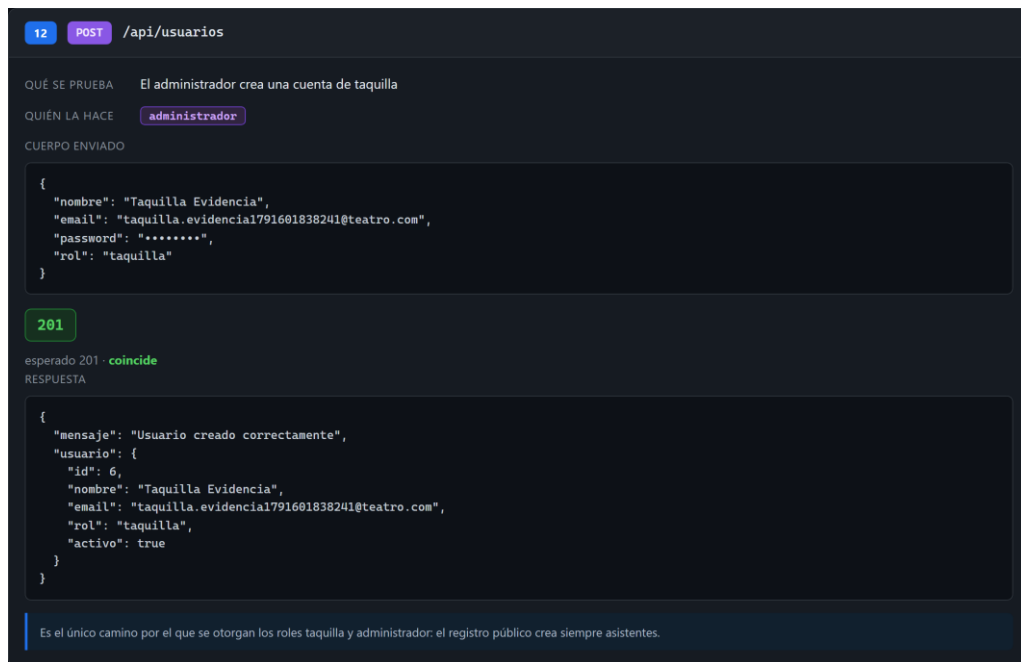


Figura 7. POST /api/usuarios como administrador: 201, se crea una cuenta de taquilla sin devolver el hash.

Los demás roles no pueden usar esa ruta. Lo importante no es si el rol pedido es válido, sino quién intenta asignarlo. La taquilla recibe 403, y un asistente que pide una cuenta de administrador también:



Figura 8. POST /api/usuarios como taquilla: 403.

El rol pasa por una lista blanca de dos valores, `taquilla` y `administrador`. Cualquier otro, como `superadmin`, `asistente` o `ADMINISTRADOR` en mayúsculas, responde 400. Y aunque el administrador intente colar `id`, `activo`, `passwordHash` o `esSuperAdmin`, el id lo pone el servidor, la cuenta queda activa y el hash sale de la contraseña real.

4. Bloque 6C: IDOR/BOLA, cada quien con lo suyo

El RBAC responde "¿tu rol puede hacer esto?", pero no basta. Un asistente tiene permiso para ver boletas, pero solo **las suyas**. Si la API solo revisa el rol, el asistente A puede

cambiar el id en la URL y ver las boletas del asistente B. Eso es IDOR (referencia directa insegura) o BOLA (autorización rota a nivel de objeto).

Atar la cuenta con su perfil

Para saber cuáles boletas son de quién, cada asistente tiene un `usuarioId` que lo une a su cuenta. Solo el administrador puede asignarlo, y se revisan tres cosas: que el usuario exista (400), que sea asistente (409) y que no esté asociado a otro perfil (409).

Las guardas de propiedad

Creamos `src/middlewares/propiedad.middleware.js` con tres guardas. La idea central está en una sola línea: el asistente propio se busca desde `req.usuario.id`, que viene del token firmado por el servidor, y **nunca** desde algo que mande el cliente.

```
17 src/middlewares/propiedad.middleware.js Líneas 26 a 101

QUÉ MUESTRA Las dos primeras guardas de propiedad y la pieza que comparten

26 // El asistente de quien presenta el token
27 // Se resuelve desde req.usuario.id, nunca desde el cuerpo ni la dirección
28 // =====
29 // Es la diferencia entre autorizar y confiar: el id del JWT viene firmado por
30 // el servidor, cualquier otro id lo escribe el cliente.
31 const asistenteDelToken = (req) => {
32   asistentesService.buscarAsistentePorUsuario(req.usuario.id);
33 }
34 // =====
35 // Debe existir usuario autenticado
36 // Si falta, la culpa es de la cadena, no del cliente
37 // =====
38 // En las rutas de hoy autenticarJWT corre antes y esto nunca salta. Está por
39 // si alguien monta una guarda sin él: sin la comprobación, leer req.usuario.rol
40 // lanzaría un TypeError y el cliente recibiría un 500 por un fallo de montaje
41 // que en realidad es un 401.
42 const sinUsuario = (req, res) => {
43   if (req.usuario) return false;
44   res.status(401).json({ mensaje: "Usuario no autenticado" });
45   return true;
46 };
47 // =====
48 // Autorizar el acceso a los datos de un asistente
49 // Para GET /api/boletas/asistente/:asistenteId
50 // =====
51 const autorizarAsistentePropio = (req, res, next) => {
52   if (sinUsuario(req, res)) return;
53   if (ROLES_DE_MOSTRADOR.includes(req.usuario.rol)) return next();
54   const propio = asistenteDelToken(req);
55   if (!propio) {
56     return res.status(403).json({ mensaje: MENSAJE_SIN_ASISTENTE });
57   }
58   // El 403 llega antes que el 404 del controlador a propósito: si respondiera
59   // 404 para los ids que no existen y 403 para los ajenos, la diferencia entre
60   // las dos respuestas diría cuántos asistentes hay registrados.
61   if (propio.id !== Number(req.params.asistenteId)) {
62     return res.status(403).json({ mensaje: MENSAJE_AJENO });
63   }
64   next();
65 };
66 // =====
67 // Autorizar el acceso a una boleta
68 // Para GET /api/boletas/:id
69 // =====
70 const autorizarAccesoBoleta = (req, res, next) => {
71   if (sinUsuario(req, res)) return;
72   const boleta = boletasService.obtenerBoletaPorId(req.params.id);
73   if (!boleta) {
74     return res.status(404).json({ mensaje: "Boleta no encontrada" });
75   }
76   if (ROLES_DE_MOSTRADOR.includes(req.usuario.rol)) return next();
77   const propio = asistenteDelToken(req);
78   if (!propio) {
79     return res.status(403).json({ mensaje: MENSAJE_SIN_ASISTENTE });
80   }
81   if (boleta.asistenteId !== propio.id) {
82     return res.status(403).json({ mensaje: MENSAJE_AJENO });
83   }
84   req.boleta = boleta;
85   next();
86 };
87 // =====
88 // El asistente propio se resuelve desde req.usuario.id, que puso autenticarJWT al verificar la firma. Nunca desde el cuerpo ni la dirección.
```

Figura 9. *propiedad.middleware.js*: el asistente propio se resuelve desde el token; las guardas comparan con el recurso pedido.

Las guardas van después de validar el id y empiezan con su propia comprobación de sesión. Al revisar encontramos que, sin esa comprobación, montar una guarda sin el JWT delante habría devuelto un error 500 donde correspondía un 401. Lo corregimos.

El ataque: cambiar el número en la URL

El asistente A consulta sus boletas por su `asistenteId` y obtiene 200:

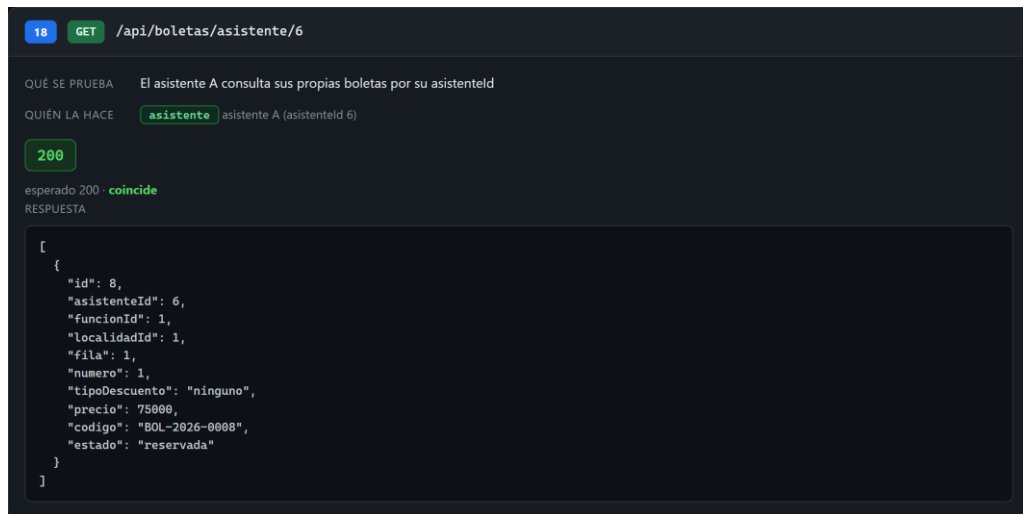


Figura 10. GET /api/boletas/asistente/6 como asistente A: 200 con su boleto.

Ahora cambia el número por el de B. Antes de este laboratorio eso habría devuelto las boletas de B. Ahora la guarda compara el perfil del token con el pedido y responde 403:

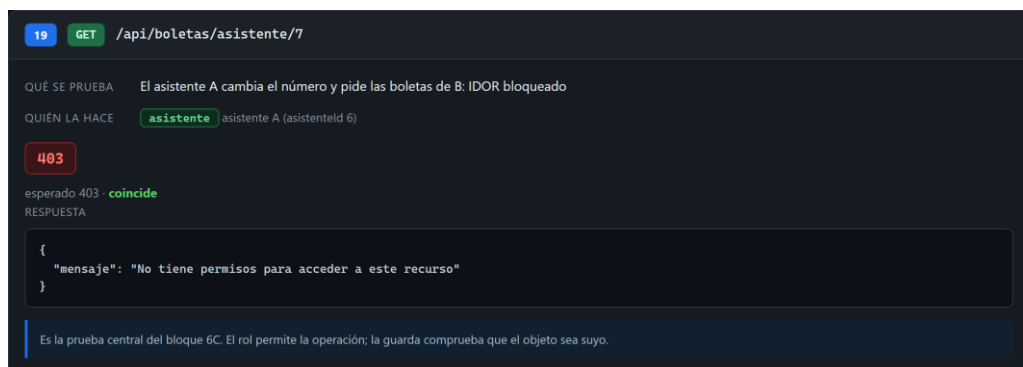


Figura 11. GET /api/boletas/asistente/7 como asistente A: 403. El IDOR queda bloqueado.

No bastaba con proteger la consulta por asistente, porque hay otra ruta: pedir una boleta directamente por su id. A puede ver su boleta 8:

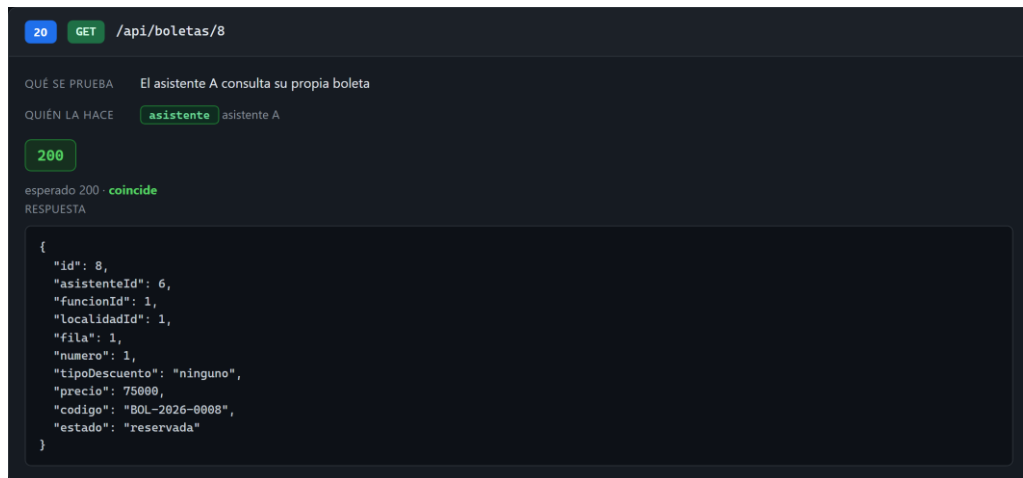


Figura 12. GET /api/boletas/8 (su propia boleta) como asistente A: 200.

Pero si pide la boleta 9, que es de B, recibe 403:

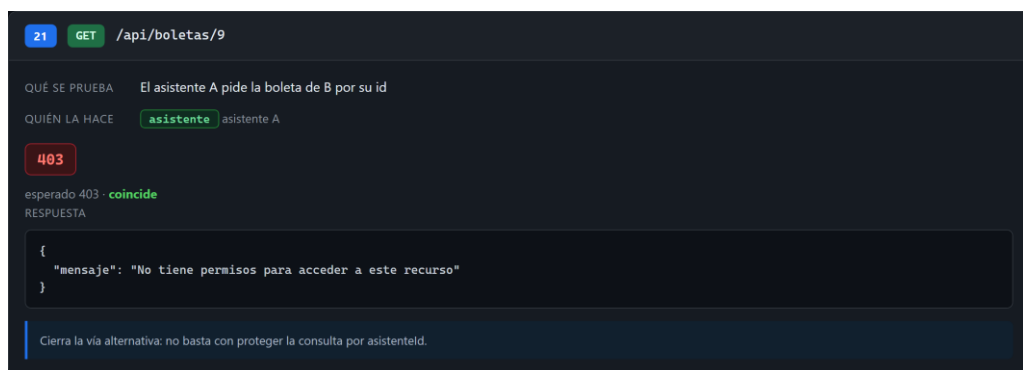


Figura 13. GET /api/boletas/9 (la boleta de B) como asistente A: 403.

Si un asistente pide un `asistenteId` que ni siquiera existe, también recibe 403 y no 404. Si respondiéramos distinto, la diferencia entre los dos códigos le diría cuántos asistentes hay registrados.

Listar todo y /mias

Listar toda la boletería es solo del administrador. El asistente recibe 403 y el administrador 200:

22 Listar toda la boletería: solo el administrador

PETICIÓN **GET** /api/boletas

QUIÉN LA HACE **asistente** asistente A

403

esperado 403 · coincide

RESPUESTA

```
{
  "mensaje": "No tiene permisos para realizar esta operación"
}
```

PETICIÓN **GET** /api/boletas

QUIÉN LA HACE **administrador**

200

esperado 200 · coincide

RESPUESTA

```
[
  {
    "id": 1,
    "asistenteId": 1,
    "funcionId": 1,
    "localidadId": 1,
    "fila": 2,
    "numero": 7,
    "tipoDescuento": "ninguno",
    "precio": 75000,
    "codigo": "BOL-2026-0001",
    "estado": "pagada"
  },
  {
    "id": 2,
    "asistenteId": 3,
    "funcionId": 1,
    "localidadId": 2,
    "fila": 4,
    "numero": 12,
    "tipoDescuento": "estudiante",
    "precio": 44000,
    "codigo": "BOL-2026-0002",
    "estado": "pagada"
  },
  {
    "id": 3,
    "asistenteId": 4,
    "funcionId": 1,
    "localidadId": 3,
    "fila": 1,
    "numero": 5,
    "tipoDescuento": "adultoMayor",
    "precio": 26600,
    "codigo": "BOL-2026-0003",
    "estado": "reservada"
  },
  {
    "id": 4,
    "asistenteId": 5,
    ... recortado para que quepa en la captura: 9 elementos, 110 líneas en total
  }
]
```

La taquilla tampoco puede: para eso están las consultas por asistente y por función.

Figura 14. GET /api/boletas: 403 para el asistente y 200 para el administrador (lista recortada en la captura).

La ruta `/mias` no lleva ningún id que se pueda cambiar: la identidad sale del token. Al asistente le devuelve solo su boleta, y al administrador le responde 403 porque esa ruta es de los asistentes, no de la administración:

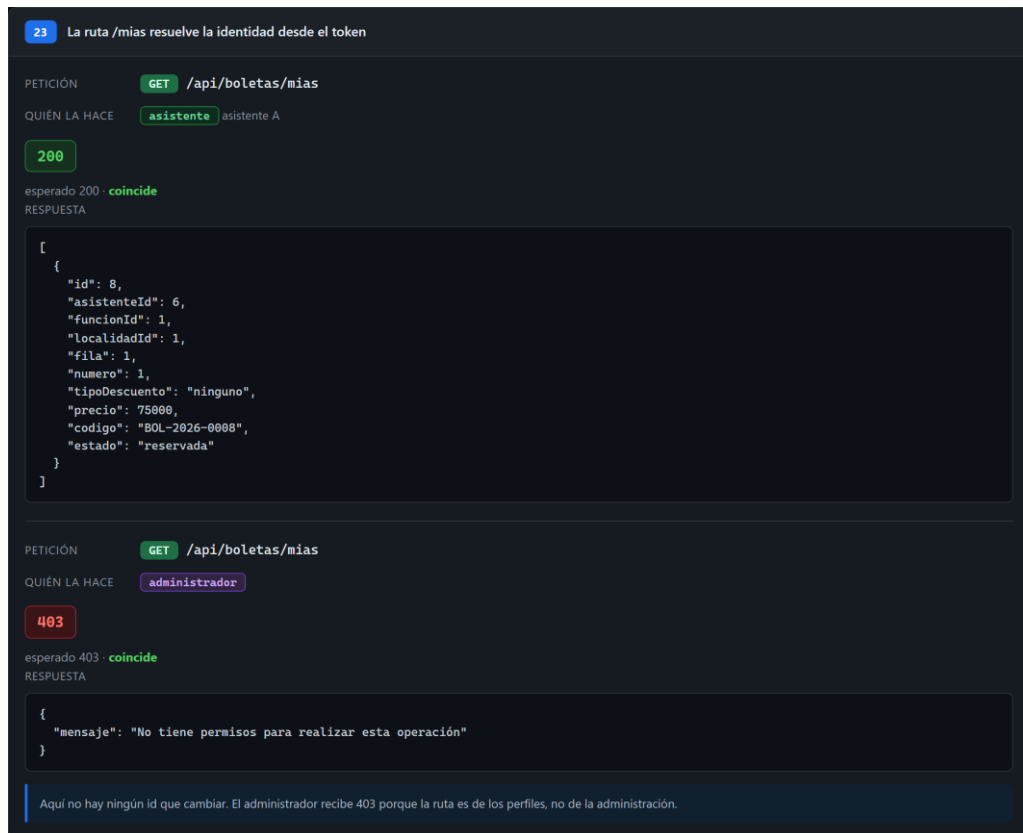


Figura 15. `GET /api/boletas/mias`: 200 solo con la boleta de A; 403 para el administrador.

Comprar a nombre de otro

El asistente sí puede comprar, porque es el uso principal del teatro. Pero ya no decide a nombre de quién. A mandó en el cuerpo el `asistenteId` de B y la boleta salió a nombre de A, porque el servidor lo toma del token. Sin esto, cambiar un número bastaría para emitir boletas a nombre de otra persona y gastarles su límite de seis.

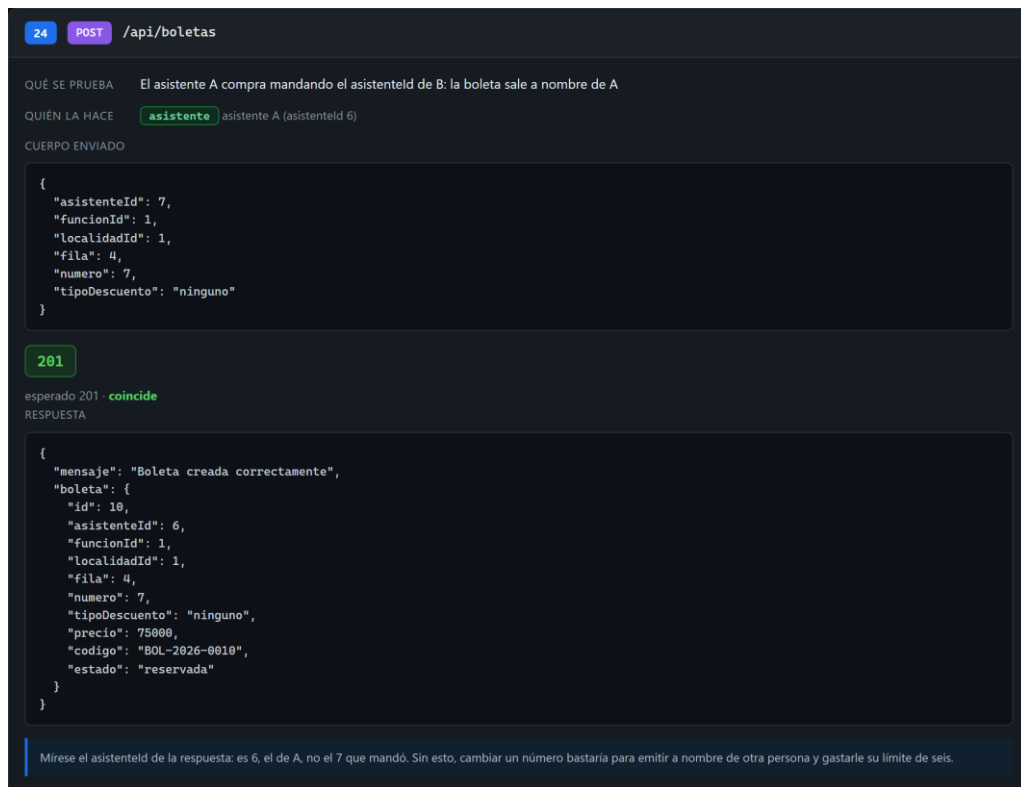


Figura 16. POST /api/boletas como A con el asistenteId de B: 201, pero la boleta queda a nombre de A (asistenteId 6).

Quién puede cambiar el estado de una boleta

Aquí el rol no alcanza y hace falta mirar el estado pedido. El asistente dueño de la boleta puede pagarla (pago simulado) o cancelarla, pero no marcarla como usada: eso sería entrar al teatro sin pasar por la puerta. Marcarla como usada es tarea de la taquilla o del administrador.

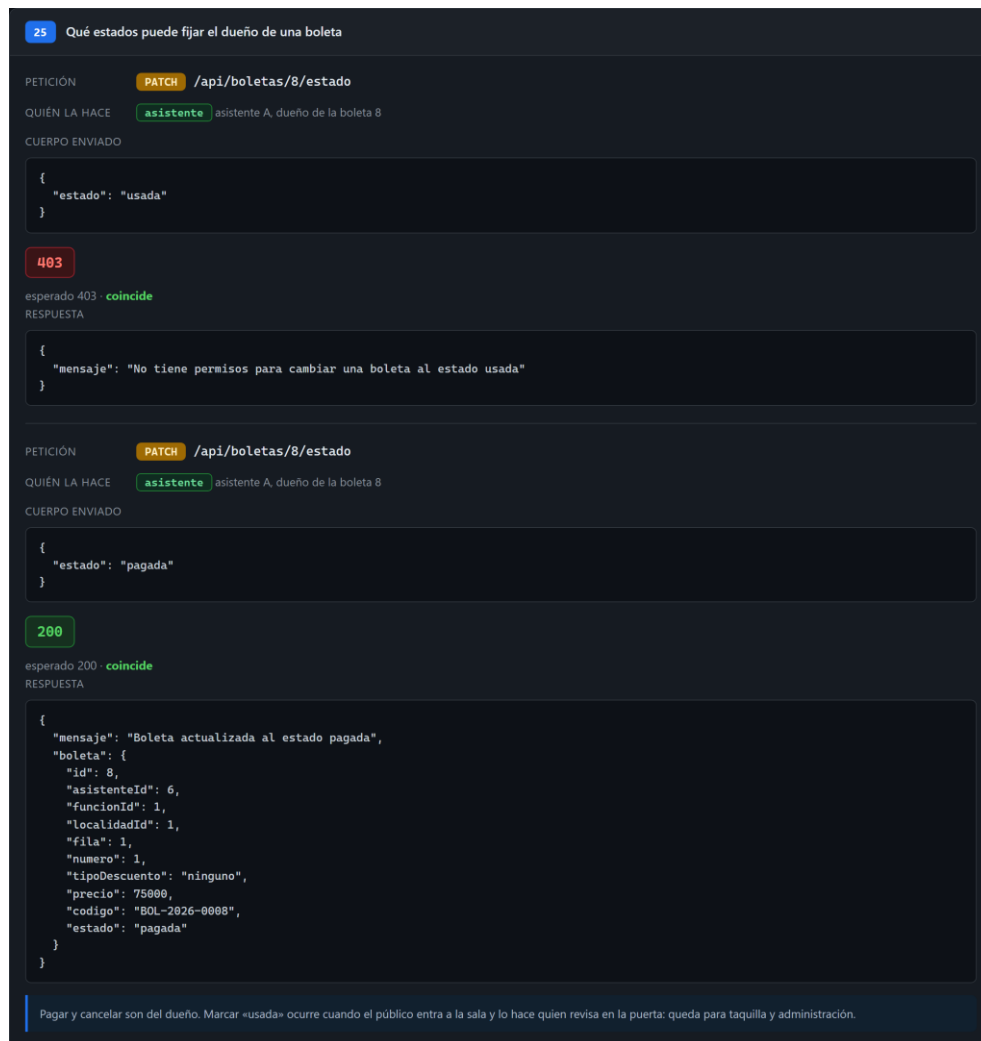


Figura 17. PATCH del estado de la boleta 8 por su dueño: "usada" da 403 y "pagada" da 200.

5. Tener permiso no salta las reglas del negocio

Ser administrador no permite violar las reglas de negocio. La autorización y las reglas son controles distintos, y se ve en el código de respuesta: el administrador sí tiene permiso, pero pasar una boleta pagada a reservada responde 409, y borrar un asistente que tiene boletas también responde 409.

Y la primera barrera sigue en pie: sin `X-API-Key` no se entra ni siquiera a la cartelera pública.

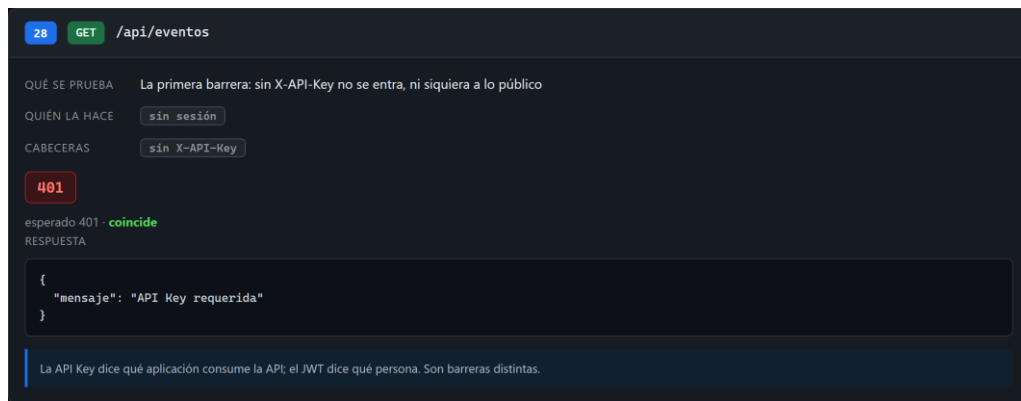


Figura 18. GET /api/eventos sin X-API-Key: 401 "API Key requerida".

6. Decisiones de seguridad que tomamos

- **El 403 va antes que el 400 o el 409.** Al asociar una cuenta, primero se revisa si el rol puede hacerlo y solo después el dato. Si no, un taquillero podría probar ids y, por la diferencia entre "no existe" (400) y "ya está asociado" (409), averiguar qué usuarios hay.
- **El 403 en vez del 404.** Pedir el `asistenteId` de alguien que no existe da 403, para que la respuesta no revele cuántos asistentes hay.
- **El `usuarioId` no se ignora en silencio.** Si la taquilla lo manda, responde 403. Ignorarlo y devolver 201 sería decir "listo" sin haber hecho el vínculo.
- **Las guardas tienen su propio 401,** para que un error de montaje no termine en un 500.
- **El `asistenteId` de una compra sale del JWT** cuando compra un asistente.
- **La cartelera es pública y el asistente puede comprar.** La guía restringe más la consulta y la escritura, pero en un teatro publicar la cartelera y vender boletas es el uso principal. Lo que protegemos es a nombre de quién se compra y quién marca una boleta como usada.
- **La taquilla no tiene boletas propias.** Es un rol que atiende a cualquier comprador, así que comprobamos que su permiso tenga límite: no lista toda la boletería, no usa `/mias`, no crea usuarios y no borra boletas.

7. Checkpoint del laboratorio

Prueba	Esperado	Obtenido	Estado
Sin JWT → asistentes	401	401	OK
JWT alterado	401	401 "Token inválido"	OK
Asistente → operación de administrador	403	403	OK
Taquilla → operación de administrador	403	403	OK
Administrador → crear cuenta de taquilla	201	201	OK
Registro con rol, activo, passwordHash y esSuperAdmin	201, asistente y activo	201, asistente y activo	OK
Taquilla → crear usuario privilegiado	403	403	OK
A → sus boletas	200	200	OK
A → boletas de B	403	403	OK
A → su boleta por id	200	200	OK
A → boleta de B por id	403	403	OK
Asistente → /mias	200, solo las suyas	200, solo la boleta 8	OK
Administrador → /mias	403	403	OK
Asistente sin perfil → /mias	403	403	OK
Administrador → todas las boletas	200	200	OK
Asistente → todas las boletas	403	403	OK
Taquilla → todas las boletas	403	403	OK
A compra con el asistenteId de B	201 a nombre de A	201, asistenteId 6	OK
Asistente → modificar o borrar boletas	403	403	OK
Asistente marca su boleta como usada	403	403	OK
Administrador → transición inválida	409	409	OK
Sin X-API-Key	401	401 "API Key requerida"	OK

8. Pruebas automáticas

Además de las figuras, escribimos la batería `pruebas/pruebas-autorizacion.js`, con 104 comprobaciones que recorren los tres bloques. Las credenciales del administrador no están escritas en las pruebas: se leen del `.env`.

```
29 La batería de autorización, con sus 104 comprobaciones

pruebas/pruebas-autorizacion.js

$ node pruebas/pruebas-autorizacion.js
== Identidades
== La puerta
== Catálogo
== Datos personales
== Rutas propias
== Usuarios administrativos
== Lista blanca del rol
== Mass Assignment
== Unicidad del correo
== Montaje de A y B
== IDOR por asistenteId
== IDOR por id de boleta
== Listado completo
== Las boletas de cada cual
== Estado de una boleta
== Perfil propio
== Asociación con una cuenta
== La puerta del GC
== Mass Assignment en el registro
== Modificar y borrar boletas
== Cierre de la tabla de la guía
== Borrado

(104 comprobaciones; se muestran los grupos y el cierre)

OK DELETE /api/boletas/9          403 Un asistente no borra una boleta
OK DELETE /api/boletas/9          403 La taquilla tampoco: borrar es solo del administrador
OK --                             sí El administrador sí pasa el control de rol del borrado

== Cierre de la tabla de la guía =====
OK GET /api/boletas/mias          401 La ruta propia también exige sesión: /mias sin JWT es 401
OK PATCH /api/boletas/8/estado    409 El administrador tampoco revive una boleta cancelada
OK DELETE /api/asistentes/1       409 El administrador no borra un asistente con boletas: 409, no 200

== Borrado =====
OK DELETE /api/eventos/12         403 Un asistente no borra el evento de prueba
OK DELETE /api/eventos/12         200 El administrador sí lo borra

Total: 104 pruebas | Correctas: 104 | Fallidas: 0

Cubre los tres bloques: RBAC, gestión de usuarios e IDOR/BOLA con dos asistentes. Ninguna credencial está escrita en el código de las pruebas: salen del .env a través de pruebas/sesion.js.
```

Figura 19. Batería de autorización: 104 comprobaciones, 0 fallos.

Para que las pruebas anteriores siguieran funcionando cuando las escrituras empezaron a pedir JWT, las baterías ahora inician sesión con el administrador inicial. No tuvimos que cambiar ni un resultado esperado. En total son 356 pruebas en nueve baterías, cada una contra un servidor recién arrancado, y las 356 pasaron.

9. Swagger, Semgrep y npm audit

Las 36 operaciones protegidas quedaron documentadas en Swagger con `ApiKeyAuth` y `BearerAuth` a la vez y con sus respuestas 401 y 403. La nueva sección Usuarios explica que es la única forma de crear cuentas de taquilla o administración. La contraseña que se ve es el ejemplo de la documentación, no una credencial.

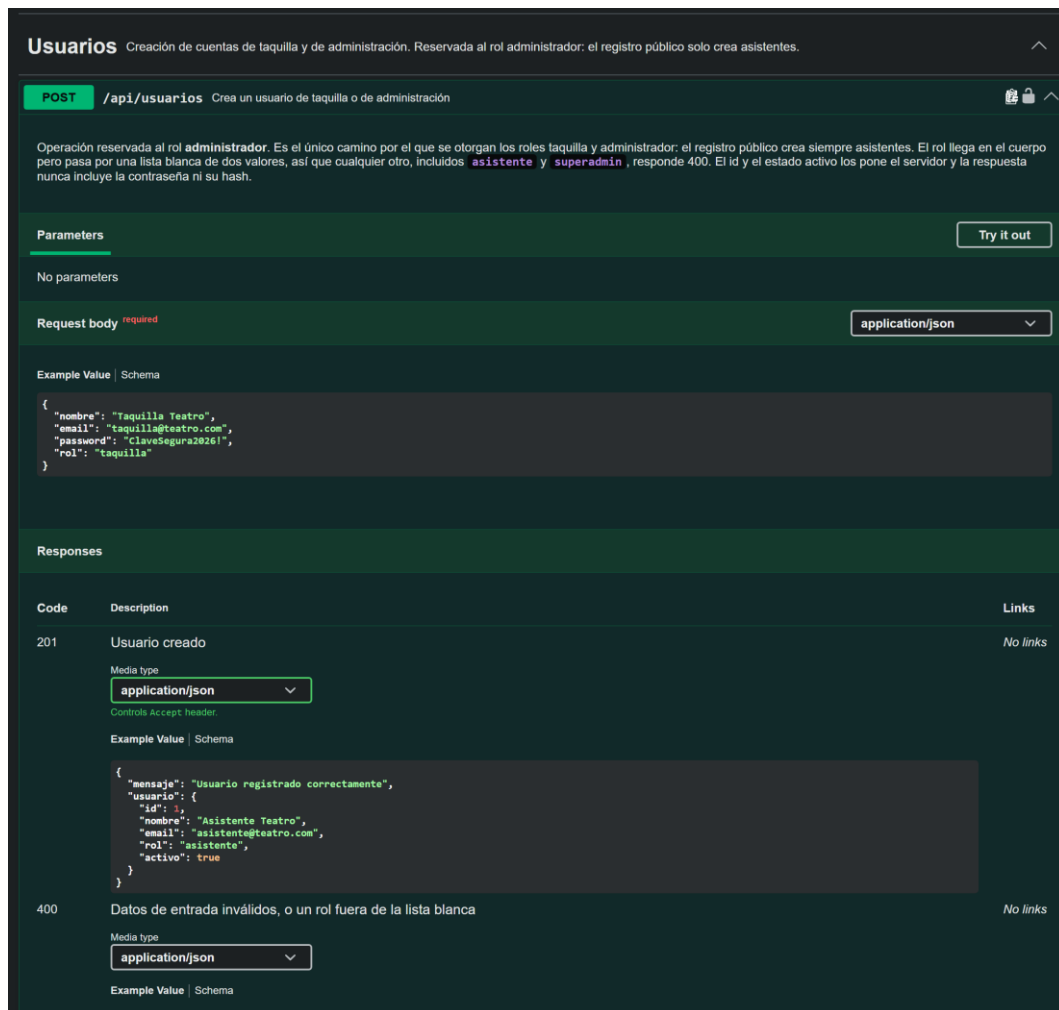


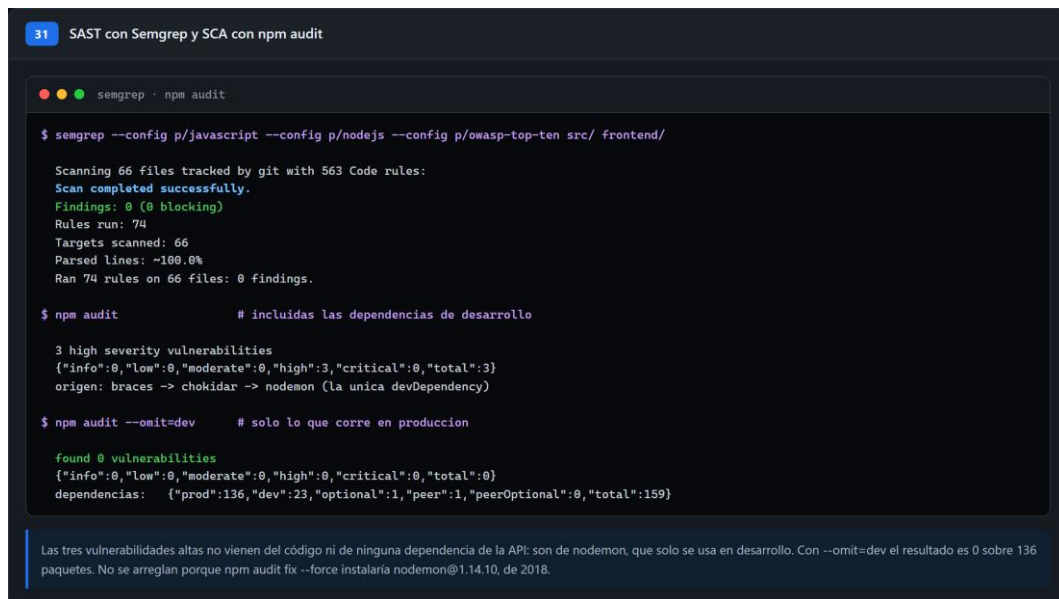
Figura 20. Swagger UI: sección Usuarios con el candado de autenticación y la descripción de la operación.

Al terminar volvimos a pasar las herramientas SAST y SCA:

Herramienta	Lab 9	Lab 10
Endpoints / rutas	42 / 22	48 / 26
Pruebas automáticas	224	356
Semgrep: hallazgos	0	0
npm audit (producción)	0	0
npm audit (con desarrollo)	0	3 altas

Por primera vez en el semestre `npm audit` no da cero, y no lo escondimos. Las tres vulnerabilidades vienen de `braces`, que llega por `nodemon`, la herramienta que reinicia el servidor mientras programamos. No se instala en producción, y por eso con `--omit=dev` el resultado es 0. No usamos `npm audit fix --force` porque bajaría `nodemon` a una versión de 2018.

Semgrep tuvo que correr con Docker, porque esta vez el Control inteligente de aplicaciones de Windows bloqueó su binario interno. No desactivamos esa protección del equipo.



```
31 SAST con Semgrep y SCA con npm audit

$ semgrep --config p/javascript --config p/nodejs --config p/owasp-top-ten src/ frontend/

Scanning 66 files tracked by git with 563 Code rules:
Scan completed successfully.
Findings: 0 (0 blocking)
Rules run: 74
Targets scanned: 66
Parsed lines: ~100.0%
Ran 74 rules on 66 files: 0 findings.

$ npm audit # incluidas las dependencias de desarrollo

3 high severity vulnerabilities
{
  "info": 0, "low": 0, "moderate": 0, "high": 3, "critical": 0, "total": 3
}
origen: braces -> chokidar -> nodemon (la unica devDependency)

$ npm audit --omit=dev # solo lo que corre en production

found 0 vulnerabilities
{
  "info": 0, "low": 0, "moderate": 0, "high": 0, "critical": 0, "total": 0
}
dependencias: { "prod": 136, "dev": 23, "optional": 1, "peer": 1, "peerOptional": 0, "total": 159 }

Las tres vulnerabilidades altas no vienen del código ni de ninguna dependencia de la API: son de nodemon, que solo se usa en desarrollo. Con --omit=dev el resultado es 0 sobre 136 paquetes. No se arreglan porque npm audit fix --force instalaría nodemon@1.14.10, de 2018.
```

Figura 21. Semgrep (74 reglas, 66 archivos, 0 hallazgos) y npm audit con y sin dependencias de desarrollo.

10. Lo que este laboratorio no resuelve

- **GET /api/boletas/:id revela qué ids existen:** responde 404 a una boleta inexistente y 403 a una ajena. Lo coherente con el resto sería responder siempre 403.
- **Un token conserva el rol con el que se emitió.** Si a alguien le cambian el rol o lo desactivan, su token viejo sigue sirviendo hasta que venza, y ahora el rol importa.
- **No se puede revocar un token** antes de que caduque.
- **La taquilla no puede iniciar ni terminar funciones;** eso pediría un control por cada transición.
- **No hay registro de auditoría:** no queda constancia de qué administrador creó qué cuenta.
- **Todo sigue en memoria.** Al reiniciar desaparecen los usuarios; el administrador inicial se vuelve a crear, los demás no.

11. Repositorio

<https://github.com/JuanK2550/API-TEATRO>

Los tres bloques quedaron en commits separados: el RBAC y el administrador inicial en 90ae622, la gestión de usuarios en e956f70 y la propiedad de asistentes y boletas en

[f70c3bf](#). El informe técnico con las salidas de Semgrep y npm audit está en [8464153](#), y las últimas correcciones a las guardas y las pruebas en [914bc6a](#).